

Test Statistics

Negative-weight shortest paths: Johnson/ALT baselines and potential-free CCH

Aarav Gupta

<https://github.com/AaravG/negative-weight-cch>

1 Summary

- 17,916 automated correctness checks, all passing.
- Every exact method matched the reference answer on every benchmark query (synthetic, NY, BAY, full USA).
- Textbook A* with a straight-line heuristic was wrong on roughly half of all queries with negative edges (it is not an exact method here).
- CCH without potentials: 400/400 queries correct on NY and BAY; partial updates identical to full re-computation in all 16 runs; all 4 injected negative cycles detected.

2 Automated correctness tests

Test	Graphs	Checks	Result
<code>test_correctness.py</code> : A* with valid heuristics (Johnson, ALT, domain bound, combined), bidirectional and 2-core search vs Bellman–Ford	12 small synthetic graphs (grid, geometric, random)	1,380	All correct
<code>test_cch.py</code> : CCH queries, query-cost bound, partial updates vs full recomputation, negative-cycle detection and removal	12 small synthetic graphs	996	All correct
<code>test_cch_battery.py</code> : battery charge functions vs step-by-step simulation	Random road sequences	15,000	All correct
<code>test_cch_battery.py</code> : battery CCH queries vs reference search	6 small graphs $\times$ 3 battery sizes	540	All correct
Total automated checks		17,916	All correct

Textbook A with a straight-line heuristic is not included in the automated tests because it is not exact with negative edges. Its error rate is reported in Section 3.*

3 Benchmark answers checked against the reference

Run	Graph	Queries	Exact methods	Textbook A*
Synthetic	EV grid 100×100	30	30/30	11/30 correct
Synthetic	EV grid 200×200	30	30/30	13/30 correct
Synthetic	EV road-like 30k	30	30/30	9/30 correct
Synthetic	Random shift 20k (no geometry)	30	30/30	n/a
DIMACS	NY, EV terrain	20	20/20	10/20 correct
DIMACS	NY, random shift	20	20/20	n/a
DIMACS	BAY, EV terrain	20	20/20	12/20 correct
DIMACS	BAY, random shift	20	20/20	n/a
Full USA (24M nodes)	EV terrain	10	10/10	5/10 correct (3 gave up)
Full USA	Random shift	10	10/10	n/a
CCH	NY and BAY, 2 weightings each	100 each	400/400	n/a
CCH updates (1, 10, 100, 1,000 edges)	NY and BAY, 2 weightings each	16×10	160/160 ; shortcut values identical in all 16	n/a
CCH negative-cycle test	NY and BAY, 2 weightings each	4	4/4 detected; clean after restore	4/4 n/a

Exact methods: Bellman–Ford, Johnson + Dijkstra, Johnson + ALT, domain bound, combined bound, bidirectional search (1 and 2 cores), and CCH. Textbook A is not applicable on random-shift graphs, which have no geometry.*

4 Synthetic graphs (potential-based methods)

Pure Python, 30 queries per graph. Nodes expanded as a share of all nodes. Break-even = number of queries after which preprocessing pays off versus Bellman–Ford.

EV grid 100×100 (10,000 nodes, 12% negative edges)

Method	Preproc. (s)	Query (ms)	Nodes expanded	Correct	Break-even
Bellman–Ford (SPFA)	0	5.48	10,004 (100%)	30/30	–
Textbook A* (straight-line)	0	8.16	8,590 (85.9%)	11/30	–
Johnson + Dijkstra	0.016	4.87	5,066 (50.7%)	30/30	25.7
Johnson + ALT (16 landmarks)	0.515	0.79	202 (2.0%)	30/30	109.8
Domain bound	0	1.66	1,265 (12.7%)	30/30	–
Combined max(ALT, domain)	0.515	1.01	195 (2.0%)	30/30	115.4
Bidirectional, 1 core	0.515	1.98	245 (2.5%)	30/30	147.3
Bidirectional, 2 cores	0.515	1.10	223 (2.2%)	30/30	117.7

EV grid 200×200 (40,000 nodes, 2% negative edges)

Method	Preproc. (s)	Query (ms)	Nodes expanded	Correct	Break-even
Bellman–Ford (SPFA)	0	26.05	40,000 (100%)	30/30	–
Textbook A* (straight-line)	0	16.10	15,256 (38.1%)	13/30	–
Johnson + Dijkstra	0.017	21.91	18,893 (47.2%)	30/30	4.2
Johnson + ALT (16 landmarks)	2.02	3.45	1,028 (2.6%)	30/30	89.4
Domain bound	0	4.22	2,877 (7.2%)	30/30	–
Combined max(ALT, domain)	2.02	4.15	1,000 (2.5%)	30/30	92.2
Bidirectional, 1 core	2.02	7.56	1,104 (2.8%)	30/30	109.3
Bidirectional, 2 cores	2.02	3.62	1,087 (2.7%)	30/30	90.1

EV road-like graph (30,000 nodes, 13% negative edges)

Method	Preproc. (s)	Query (ms)	Nodes expanded	Correct	Break-even
Bellman–Ford (SPFA)	0	44.61	100,946 (336.5%)	30/30	–
Textbook A* (straight-line)	0	73.39	84,205 (280.7%)	9/30	–
Johnson + Dijkstra	0.116	15.33	13,513 (45.0%)	30/30	3.9
Johnson + ALT (16 landmarks)	1.429	3.24	709 (2.4%)	30/30	34.5
Domain bound	0	6.00	4,534 (15.1%)	30/30	–
Combined max(ALT, domain)	1.429	3.88	699 (2.3%)	30/30	35.1
Bidirectional, 1 core	1.429	7.06	713 (2.4%)	30/30	38.0
Bidirectional, 2 cores	1.429	3.74	744 (2.5%)	30/30	35.0

Random shift, no geometry (20,000 nodes, 35% negative edges)

Method	Preproc. (s)	Query (ms)	Nodes expanded	Correct	Break-even
Bellman–Ford (SPFA)	0	17.62	40,985 (204.9%)	30/30	–
Johnson + Dijkstra	0.011	13.49	10,162 (50.8%)	30/30	2.7
Johnson + ALT (16 landmarks)	0.93	7.57	633 (3.2%)	30/30	92.5
Bidirectional, 1 core	0.93	5.31	333 (1.7%)	30/30	75.5
Bidirectional, 2 cores	0.93	2.31	289 (1.4%)	30/30	60.7

5 DIMACS road graphs (potential-based methods)

New York: 264,346 nodes, 733,846 edges. SF Bay Area: 321,270 nodes, 800,172 edges. 20 queries per graph. EV terrain = real road lengths plus synthetic elevation (10% negative edges); random shift = 20% negative edges.

NY, EV terrain

Method	Preproc. (s)	Query (ms)	Nodes expanded	vs BF	Correct
Bellman–Ford (SPFA)	0	10,235.6	16,584,555 (6,274%)	1.0×	20/20
Textbook A* (straight-line)	0	2,283.3	2,892,200 (1,094%)	4.5×	10/20
Johnson + Dijkstra	2.44	152.1	135,169 (51.1%)	67×	20/20
Johnson + ALT (16 landmarks)	11.89	17.0	5,773 (2.2%)	603×	20/20
Domain bound	0	65.4	48,864 (18.5%)	157×	20/20
Combined max(ALT, domain)	11.89	20.8	5,700 (2.2%)	491×	20/20
Bidirectional, 1 core	11.89	22.1	3,865 (1.5%)	463×	20/20
Bidirectional, 2 cores	11.89	10.0	3,455 (1.3%)	1,021×	20/20

NY, random shift

Method	Preproc. (s)	Query (ms)	Nodes expanded	vs BF	Correct
Bellman–Ford (SPFA)	0	10,543.7	17,074,664 (6,459%)	1.0×	20/20
Johnson + Dijkstra	0.18	140.3	128,207 (48.5%)	75×	20/20
Johnson + ALT (16 landmarks)	9.39	18.5	5,926 (2.2%)	571×	20/20
Bidirectional, 1 core	9.39	16.4	3,626 (1.4%)	644×	20/20
Bidirectional, 2 cores	9.39	8.5	3,692 (1.4%)	1,248×	20/20

BAY, EV terrain

Method	Preproc. (s)	Query (ms)	Nodes expanded	vs BF	Correct
Bellman–Ford (SPFA)	0	13,983.4	22,359,674 (6,960%)	1.0×	20/20
Textbook A* (straight-line)	0	286.2	417,823 (130%)	48.9×	12/20
Johnson + Dijkstra	2.78	143.5	140,441 (43.7%)	97×	20/20
Johnson + ALT (16 landmarks)	13.43	26.9	9,632 (3.0%)	521×	20/20
Domain bound	0	70.7	58,280 (18.1%)	198×	20/20
Combined max(ALT, domain)	13.43	33.3	9,616 (3.0%)	420×	20/20
Bidirectional, 1 core	13.43	22.8	4,187 (1.3%)	613×	20/20
Bidirectional, 2 cores	13.43	12.3	4,346 (1.4%)	1,140×	20/20

BAY, random shift

Method	Preproc. (s)	Query (ms)	Nodes expanded	vs BF	Correct
Bellman–Ford (SPFA)	0	14,500.3	23,611,873 (7,350%)	1.0×	20/20
Johnson + Dijkstra	0.21	154.4	145,233 (45.2%)	94×	20/20
Johnson + ALT (16 landmarks)	11.22	36.6	11,641 (3.6%)	396×	20/20
Bidirectional, 1 core	11.22	37.7	8,542 (2.7%)	385×	20/20
Bidirectional, 2 cores	11.22	13.1	6,227 (1.9%)	1,109×	20/20

Nodes expanded above 100% for Bellman–Ford means nodes were processed repeatedly (about 60–70 times each on average).

6 Full USA road graph (potential-based methods)

23,947,347 nodes, 58,333,344 edges. 10 queries per method, timed one at a time on a warm cache. Bellman–Ford was stopped after more than 35 minutes per query.

Preprocessing	EV terrain (10% negative)			Random shift (20% negative)		
Johnson’s step (Bellman–Ford)	34.2 s			17.8 s		
8 ALT landmarks (16 Dijkstra runs in parallel)	93.7 s wall (526 s summed)			61.0 s wall (520 s summed)		

Method	EV terrain			Random shift		
	Query (s)	Nodes exp.	Correct	Query (s)	Nodes exp.	Correct
Johnson + Dijkstra	11.89	12,832,909	10/10	9.25	12,670,325	10/10
Domain bound	10.94	9,545,337	10/10	–	–	–
Textbook A* (straight-line)	12.12	10,421,359	5/10	–	–	–
Johnson + ALT (8 landmarks)	3.67	1,330,732	10/10	1.88	916,938	10/10
Combined max(ALT, domain)	4.14	1,329,583	10/10	–	–	–
Bidirectional, 1 core	2.81	499,533	10/10	1.55	410,556	10/10
Bidirectional, 2 cores	1.17	514,677	10/10	0.83	428,622	10/10
Bellman–Ford (SPFA)	>2,100	–	–	>2,100	–	–

7 CCH without potentials (NY and BAY)

Metric-independent structure (built once per road network)

Graph	Ordering	Contr. + triangles	Shortcut arcs	Triangles	Tree depth
NY	202 s	2.5 s	1,529,403	10,057,336	344
BAY	183 s	1.6 s	1,242,312	5,171,378	247

A simple geometric ordering on NY gave 4,466,000 arcs, 298,852,070 triangles and depth 1,236; the inertial-flow ordering cut triangles about 30×.

Preparing a new set of weights

Graph / weighting	Johnson's step	+ 16 ALT landmarks	CCH customization
NY, EV terrain	3.0 s	9.5 s	1.6 s
NY, random shift	0.2 s	8.9 s	1.6 s
BAY, EV terrain	3.6 s	10.6 s	0.9 s
BAY, random shift	0.3 s	10.2 s	1.0 s

The CCH customization time includes the negative-cycle check.

Query time (same 30 query pairs for all methods; CCH on 100 pairs)

Method	NY terrain	NY shift	BAY terrain	BAY shift
Johnson + Dijkstra	116.7 ms	128.0 ms	183.3 ms	187.3 ms
Johnson + ALT (16 landmarks)	21.4 ms	23.1 ms	44.9 ms	42.7 ms
Bidirectional ALT, 1 core	14.3 ms	23.7 ms	36.7 ms	24.8 ms
CCH, no potential	0.77 ms	0.76 ms	0.31 ms	0.30 ms
CCH speed-up vs Johnson + Dijkstra	151×	169×	599×	622×
Correct (CCH)	100/100	100/100	100/100	100/100

Query cost versus the depth bound (200 random queries)

Graph	Tree depth d	Bound $d(d+1)$	Arcs scanned: mean	Arcs scanned: max
NY	344	118,680	11,683	29,036
BAY	247	61,256	4,778	10,798

Partial updates (k random edges made more expensive)

Graph / weighting	k	Partial update	Arcs re-eval.	Full custom.	Redo Johnson + ALT	Correct
NY, terrain	1	0.1 ms	57	1.6 s	11.5 s	10/10
NY, terrain	10	25.5 ms	4,524	1.6 s	11.7 s	10/10
NY, terrain	100	124.7 ms	23,340	1.6 s	11.8 s	10/10
NY, terrain	1,000	645.4 ms	166,538	1.6 s	11.7 s	10/10
NY, shift	1	0.3 ms	116	1.6 s	9.5 s	10/10
NY, shift	10	16.4 ms	2,774	1.6 s	9.5 s	10/10
NY, shift	100	138.3 ms	27,873	1.6 s	9.4 s	10/10
NY, shift	1,000	627.2 ms	161,138	1.6 s	9.5 s	10/10
BAY, terrain	1	0.6 ms	193	0.9 s	13.2 s	10/10
BAY, terrain	10	9.5 ms	2,513	0.9 s	13.3 s	10/10
BAY, terrain	100	47.6 ms	13,187	0.9 s	13.4 s	10/10
BAY, terrain	1,000	270.6 ms	90,304	0.9 s	13.4 s	10/10
BAY, shift	1	0.0 ms	1	0.9 s	10.9 s	10/10
BAY, shift	10	5.6 ms	1,561	0.9 s	10.8 s	10/10
BAY, shift	100	29.9 ms	9,386	0.9 s	10.8 s	10/10
BAY, shift	1,000	249.4 ms	85,384	1.0 s	11.0 s	10/10

In every update run, the shortcut values after the partial update were identical to a full re-customization. One-time update index: 4.7 s (NY), 2.6 s (BAY).

Negative-cycle detection (a 2-cycle of weight -10^9 injected by a partial update)

Graph / weighting	CCH detects	CCH time	BF detects	BF time	Clean after restore
NY, terrain	Yes	0.07 s	Yes	0.1 s	Yes
NY, shift	Yes	0.07 s	Yes	0.2 s	Yes
BAY, terrain	Yes	0.03 s	Yes	0.1 s	Yes
BAY, shift	Yes	0.03 s	Yes	0.2 s	Yes

CCH time includes the partial update and the scan. Checking only the arcs changed by the update: NY 2 arcs (<0.01 ms) versus 64 ms for a full scan; BAY 1 arc (<0.01 ms) versus 29 ms.

8 Battery-constrained CCH (small graphs only)

Battery capacity M ; each road maps charge b to $\min(\text{out}, b - \text{cost})$ when $b \geq \text{in}$. 30 random queries per graph and capacity, with random starting charge. All answers matched the reference search.

Graphs	M	Largest profile	Mean profile size	Infeasible (of 30)	Correct
3 EV grids (144 nodes)	3	1–3	0.52–0.57	26–29	90/90
3 EV grids (144 nodes)	10	1–3	0.95–0.98	22–24	90/90
3 EV grids (144 nodes)	40	1–4	1.00–1.04	5	90/90
3 geometric graphs (250 nodes)	3	2–5	0.62–0.68	26–29	90/90
3 geometric graphs (250 nodes)	10	2–5	0.94–0.97	22–28	90/90
3 geometric graphs (250 nodes)	40	2–5	1.01–1.07	6–12	90/90

Plus 15,000 checks of the charge-function algebra against step-by-step simulation. The NY/BAY battery benchmark has not been run to completion yet.

9 Notes

- All times are from a pure-Python prototype on one machine (32 logical cores, 31 GB RAM); compiled implementations would be much faster.
- Section 7 and Section 5 times come from different runs with different random query pairs, so compare within a table rather than across tables.
- The full-USA timings in Section 6 are from a clean re-run with one query at a time.
- Elevation in the EV weighting is synthetic; real elevation data has not been used yet.